Savitribai Phule Pune University

A PROJECTREPORT

ON

AI-POWERED ATTENDANCE SYSTEM: INTELLIGENT SMART CLASSROOM MANAGEMENT (EDU-VISION)

Submitted by

Atharva Badgujar	**Seat No. B400320234**
Vedant Bhosale	**Seat No. B400320241**
Parth Deokar	**Seat No. B400320267**
Swanand Joshi	**Seat No. B400320300**

Under the guidance of

Prof. (Mr.) Anand D. Dhawale

Department of Computer Engineering

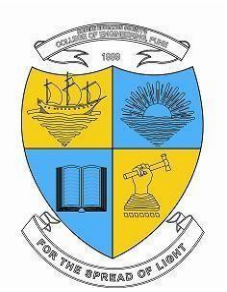
Modern Education Society's Wadia College Engineering,

19, Bund Garden, V. K. Joag Path, Pune-411 001

[2025-26]

Modern Education Society's Wadia College Engineering,

19, Bund Garden, V. K. Joag Path, Pune-411 001



C E R T I F I C A T E

This is to certify that the Project Entitled

"AI-POWERED ATTENDANCE SYSTEM: INTELLIGENT SMART CLASSROOM MANAGEMENT (EDU-VISION)"

Submitted by

Atharva Badgujar	Exam No: B400320234
Vedant Bhosale	Exam No: B400320241
Parth Deokar	Exam No: B400320267
Swanand Joshi	Exam No: B400320300

is a bonafide work carried out by Students under the supervision of Prof. Anand. D. Dhawale and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Computer Engineering).

Prof. (Mr.) A. D. Dhawale
Project Guide
(Dept. of Computer Engg.)

Dr. (Mrs.) N. F. Shaikh
H.O.D.
(Dept of Computer Engg.)

(Dr. M. P. Dale)
Principal,
MES Wadia College of Engineering Pune —01

Sign of Internal Examiner

Sign of External Examiner

Place: Pune
Date:

PROJECT APPROVAL SHEET

AI-POWERED ATTENDANCE SYSTEM: INTELLIGENT SMART CLASSROOM MANAGEMENT (EDU-VISION)

Is successfully completed by

Atharva Badgujar	Exam No: B400320234
Vedant Bhosale	Exam No: B400320241
Parth Deokar	Exam No: B400320267
Swanand Joshi	Exam No: B400320300

at

DEPARTMENT OF COMPUTER ENGINEERING

Modern Education Society's Wadia College of Engineering

SAVITRIBAI PHULE PUNE UNIVERSITY,PUNE

ACADEMIC YEAR 2025-2026

Prof. A.D.Dhawale Internal Guide Dept. of Computer Engg.	Dr (Mrs.) N.F.Shaikh H.O.D Dept. of Computer Engg.

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on **AI-POWERED ATTENDANCE SYSTEM: INTELLIGENT SMART CLASSROOM MANAGEMENT (EDU-VISION)**.

We would like to take this opportunity to express our sincere gratitude to our internal guide **Prof. A.D. Dhawale** for providing us with invaluable guidance, constant motivation, and timely feedback throughout the course of this project. His deep technical insight and encouragement were instrumental in shaping this work.

We are also deeply grateful to **Dr. (Mrs.) N. F. Shaikh**, Head of the Computer Engineering Department, MES Wadia College of Engineering, for providing us with the necessary infrastructure and academic support throughout the project duration.

We extend our special thanks to **Dr. M. P. Dale**, Principal, MES Wadia College of Engineering for fostering an environment that encourages innovation and research oriented projects.

We are thankful to the faculty and staff of the Department of Computer Engineering for their continuous support and for making available the computational laboratories equipped with the necessary hardware and software platforms required for the development and testing of this project.

Finally, we thank our families and peers for their unwavering moral support throughout this academic journey.

Atharva Badgujar
Vedant Bhosale
Parth Deokar
Swanand Joshi
(B.E Computer Engineering)

ABSTRACT

The EDU-VISION AI-Powered Attendance System is a locally deployable, desktop and API-driven application designed to automate student attendance tracking using real-time facial recognition technology. Traditional attendance methods—manual roll-calls or card-based systems—are time-consuming, error-prone, and susceptible to proxy attendance. EDU-VISION addresses all these challenges through an intelligent, contactless, and fully automated pipeline built on state-of-the-art computer vision and deep learning models.

The system is engineered around the InsightFace buffalo_l deep learning model, which generates 512-dimensional face embeddings with high accuracy across varying lighting and pose conditions. An optional YOLOv8 detection stage accelerates face localisation before embedding. All enrolled student encodings are loaded into RAM at startup, enabling in-memory Euclidean distance matching at real-time frame rates without repeated database round-trips.

EDU-VISION supports two attendance recording modes. In **Kiosk Mode**, students are automatically recognised as they walk past a fixed camera; a per-student cooldown timer prevents duplicate records within a configurable window. In **Classroom Session Mode**, a teacher initiates a timed session and the system bins detections across one-minute intervals; a student is marked "Present" only when detected in at least 70% of those bins, thereby capturing sustained presence rather than a momentary appearance.

The backend is powered by FastAPI and PostgreSQL, with eight normalised tables managing students, sessions, detections, and attendance logs. A React-accessible REST API enables remote frame processing from web and mobile frontends. The system additionally integrates automated email notifications for attendance reports and low-attendance warnings, and an AI Chat Assistant endpoint that translates natural language queries into read-only SQL using a Large Language Model, returning human-readable answers about attendance records.

The outcome is a robust, institution-deployable smart classroom management solution that reduces administrative effort, eliminates proxy attendance, and provides actionable attendance analytics through a conversational interface.

Contents

List of Figures

List of Tables

Chapter 1

SYNOPSIS

1.1 PROJECT TITLE

AI-POWERED ATTENDANCE SYSTEM: INTELLIGENT SMART CLASSROOM MANAGEMENT (EDU-VISION)

1.2 PROJECT OPTION

Internal Project — developed independently by the student group under the supervision of the internal guide, aimed at solving a practical problem in academic institutions using artificial intelligence and computer vision.

1.3 INTERNAL GUIDE

Prof. Anand. D. Dhawale
Department of Computer Engineering, MES Wadia College of Engineering

1.4 SPONSORSHIP AND EXTERNAL GUIDE

No external sponsorship. The project is entirely self-funded and internally guided.

1.5 TECHNICAL KEYWORDS (AS PER ACM KEYWORDS)

1. I. Computing Methodologies
    (a) I.4 IMAGE PROCESSING AND COMPUTER VISION
        i. I.4.1 Digitization and Image Capture
        ii. I.4.7 Feature Measurement — Feature representation
        iii. I.4.8 Scene Analysis — Object recognition
    (b) I.5 PATTERN RECOGNITION
        i. I.5.1 Models — Neural nets
        ii. I.5.4 Applications — Computer vision
2. H. Information Systems
    (a) H.2 DATABASE MANAGEMENT
        i. H.2.4 Systems — Relational databases
3. C. Computer Systems Organization
    (a) C.2 COMPUTER-COMMUNICATION NETWORKS
        i. C.2.4 Distributed Systems — Client/server, Distributed applications

1.6 PROBLEM STATEMENT

Manual attendance tracking in academic institutions is inefficient, error-prone, and vulnerable to proxy attendance. Existing RFID or barcode systems require physical tokens, which can be shared or forgotten. There is a need for a contact-free, automated, and

intelligent attendance system that can accurately identify students using facial biometrics, operate in real-time classroom conditions, handle varying lighting and pose, record sustained presence rather than momentary appearance, and provide administrators with conversational access to attendance analytics.

1.7 ABSTRACT

EDU-VISION is a locally deployable AI-powered attendance system built on InsightFace deep learning embeddings and FastAPI. It offers two recording modes—Kiosk and Classroom Session—and integrates PostgreSQL for persistence. The system eliminates manual roll-calls, prevents proxy attendance, and provides an AI Chat Assistant for natural language querying of attendance data. Automated email notifications alert students and administrators about attendance status and low-attendance warnings. The desktop frontend is built with Tkinter, and a REST API enables integration with web and mobile clients.

1.8 GOALS AND OBJECTIVES

- Automate student attendance using real-time facial recognition with ≥ 95% identification accuracy.
- Eliminate proxy attendance through individual biometric verification.
- Provide two flexible attendance modes: Kiosk (unmonitored) and Classroom Session (teacher-initiated).
- Maintain a normalised PostgreSQL database with complete audit trails for sessions, detections, and attendance logs.
- Expose a RESTful API enabling remote frame processing from web and mobile frontends.
- Deliver automated email notifications for session-end attendance reports and low-attendance warnings.

- Integrate an AI Chat Assistant that translates natural language queries to SQL and returns human-readable attendance answers.
- Ensure the system runs fully on local hardware without dependence on cloud services for core recognition.

1.9 RELEVANT MATHEMATICS ASSOCIATED WITH THE PROJECT

System Description:

- **Input:** Video frames from webcam or RTSP stream; registered student images; natural language text queries.
- **Output:** Attendance records (Present/Absent) per student per session; email notifications; SQL-derived natural language answers.
- **Face Embedding (InsightFace):** For a face image I, the model f_θ produces a 512-dimensional embedding vector: $\mathbf{e} = f_\theta(I)$, where $\mathbf{e} \in \mathbb{R}^{512}$.
- **L2 Normalisation:** Each embedding is normalised: $\hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2}$
- **Euclidean Distance Matching:** For a detected face embedding $\hat{\mathbf{e}}_d$ and a registered embedding $\hat{\mathbf{e}}_i$:

$$d_i = \|\hat{\mathbf{e}}_d - \hat{\mathbf{e}}_i\|_2 = \sqrt{\sum_{k=1}^{512} (\hat{e}_{d,k} - \hat{e}_{i,k})^2}$$

  A match is declared if $d_i < \tau$ (threshold, e.g. $\tau = 0.8$), and the identity is $\text{argmin}_i\, d_i$.
- **Blur Detection (Laplacian Variance):** $\text{Var}(\nabla^2 I) = \sigma_L^2$. If $\sigma_L^2 < \lambda_{min}$, the frame is rejected as blurry.
- **Session Presence Ratio:** Let B = total 1-minute bins in a session, b_s = bins in which student s was detected:

$$P_s = \frac{b_s}{B} \times 100\%$$

  Student s is marked "Present" if $P_s \geq 70\%$.
- **Success Condition:** $d_i < 0.8$ and $P_s \geq 70\%$.

- **Failure Condition:** $d_i \geq 0.8$ (unknown face) or $\sigma_L^2 < \lambda_{min}$ (blurry frame) or $P_s <$ 70% (insufficient presence).

1.10 NAMES OF CONFERENCES / JOURNALS WHERE PAPERS CAN BE PUBLISHED

- IEEE International Conference on Image Processing (ICIP)
- IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)
- International Conference on Pattern Recognition (ICPR)
- ACM Multimedia Conference
- SPPU-affiliated National Conference on Emerging Trends in Computer Engineering
- International Journal of Computer Vision (IJCV) — Springer

1.11 REVIEW OF CONFERENCE/JOURNAL PAPERS SUPPORTING PROJECT IDEA

1. Deng et al., "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," CVPR 2019 — proposes an additive angular margin loss that significantly improves the discriminative power of face embeddings; foundational to InsightFace.
2. Redmon & Farhadi, "YOLOv3: An Incremental Improvement," arXiv 2018 — establishes the YOLO architecture for real-time object detection adopted in the YOLOv8 pipeline used for face localisation.
3. Wang et al., "InsightFace: 2D and 3D Face Analysis Project," arXiv 2021 — describes the buffalo_l model and its superior benchmark performance on LFW, CFP-FP, and AgeDB-30 datasets.
4. Schroff et al., "FaceNet: A Unified Embedding for Face Recognition and Clustering," CVPR 2015 — pioneers the triplet-loss approach to face embeddings, motivating modern cosine/Euclidean distance matching.

5. Zhang et al., “MTCNN: Joint Face Detection and Alignment using Multi-task Cascaded CNNs,” IEEE Signal Processing Letters 2016 — a multi-stage CNN approach for face detection used as baseline for comparison.

6. Ravi et al., “A Survey on Face Recognition Attendance Systems,” International Journal of Computer Applications 2020 — reviews existing academic attendance systems and their accuracy benchmarks.

7. Bochkovskiy et al., “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv 2020 — details CSP-based architecture improvements adopted in YOLOv8.

8. Patel et al., “Automated Attendance System using Deep Learning,” ICCIT 2021 — demonstrates classroom deployment of CNN-based attendance with 93.7% accuracy.

9. He et al., “Deep Residual Learning for Image Recognition,” CVPR 2016 — ResNet architecture underpins the backbone of InsightFace models.

10. Liu et al., “SphereFace: Deep Hypersphere Embedding for Face Recognition,” CVPR 2017 — angular softmax loss serving as a precursor to ArcFace embeddings used in this project.

1.12 PLAN OF PROJECT EXECUTION

The project was executed using an iterative development model across two academic terms. Key milestones include: database schema design, engine integration (InsightFace + YOLOv8), registration module, kiosk mode, session mode, REST API, email integration, AI chat endpoint, dashboard UI, and testing. Refer Annexure C for the detailed project planner with Gantt chart.

Chapter 2

TECHNICAL KEYWORDS

2.1 AREA OF PROJECT

Artificial Intelligence, Computer Vision, Biometric Systems, Smart Classroom Technology, Web APIs.

2.2 TECHNICAL KEYWORDS

1. I. Computing Methodologies
   (a) I.4 IMAGE PROCESSING AND COMPUTER VISION
      i. I.4.1 Digitization and Image Capture — real-time webcam and RTSP stream ingestion
      ii. I.4.7 Feature Measurement — 512-dimensional face embedding vectors
      iii. I.4.8 Scene Analysis — multi-face detection and identification per frame
   (b) I.5 PATTERN RECOGNITION
      i. I.5.1 Models — InsightFace (buffalo l) deep neural network; YOLOv8 detection model
      ii. I.5.2 Design Methodology — Euclidean distance matching with L2-normalised embeddings
      iii. I.5.4 Applications — automated student attendance; proxy prevention
   (c) I.2 ARTIFICIAL INTELLIGENCE

i. I.2.7 Natural Language Processing — LLM-powered Text-to-SQL for AI Chat Assistant

2. H. Information Systems
   (a) H.2 DATABASE MANAGEMENT
      i. H.2.3 Languages — SQL; psycopg2 connection pooling
      ii. H.2.4 Systems — PostgreSQL relational DBMS; eight-table normalised schema
3. C. Computer Systems Organization
   (a) C.2 COMPUTER-COMMUNICATION NETWORKS
      i. C.2.4 Distributed Systems — FastAPI REST backend; client/server architecture
4. D. Software
   (a) D.2 SOFTWARE ENGINEERING
      i. D.2.2 Design Tools and Techniques — Tkinter GUI; FastAPI; Uvicorn ASGI server

Chapter 3

INTRODUCTION

3.1 PROJECT IDEA

EDU-VISION is conceived as a fully local, AI-powered attendance management platform for educational institutions. The central idea is to replace manual or token-based attendance with biometric facial recognition that is contactless, accurate, and automatic. The system runs entirely on institutional hardware—no cloud dependency for core recognition—while also exposing a REST API for integration with existing campus web and mobile portals. Two complementary attendance modes (Kiosk and Classroom Session) serve different institutional contexts, from unmonitored corridor cameras to teacher-supervised classroom recording.

3.2 MOTIVATION OF THE PROJECT

- Manual attendance consumes 5–10 minutes of every lecture and is universally disliked by both faculty and students.
- Proxy attendance—where one student marks another present—is a persistent academic integrity issue that existing systems fail to solve without physical intervention.
- RFID and QR-based systems require dedicated hardware tokens or smartphones, creating dependency and failure points.
- Modern deep learning face recognition models (ArcFace/InsightFace) now achieve

> 99% accuracy on LFW benchmarks, making them practically viable for campus deployment.

- Institutions increasingly demand digital, queryable attendance records for regulatory compliance; an AI Chat interface makes data accessible to non-technical administrators.
- The project provides hands-on integration of multiple advanced technologies—computer vision, REST APIs, relational databases, email automation, and LLMs—in a single production-oriented codebase.

3.3 LITERATURE SURVEY

- **FaceNet (Schroff et al., 2015):** Introduced the concept of mapping face images into a compact Euclidean embedding space such that faces of the same person cluster together. This work directly motivates the L2-distance matching strategy used in EDU-VISION.
- **ArcFace (Deng et al., 2019):** Improved upon FaceNet by introducing additive angular margin loss, achieving state-of-the-art on LFW (99.83%), CFP-FP (98.27%), and AgeDB-30 (98.28%). InsightFace buffalo_l is trained with ArcFace loss, which is why EDU-VISION adopts it.
- **YOLOv8 (Ultralytics, 2023):** The eighth generation of the YOLO object detection family, offering sub-millisecond face detection at high resolutions. Used as the optional fast detection front-end before embedding.
- **CLAHE (Zuiderveld, 1994):** Contrast Limited Adaptive Histogram Equalization normalises local contrast in captured frames, improving recognition in poorly-lit classrooms — applied as a pre-processing step before embedding.
- **Text-to-SQL with LLMs:** Recent work (Rajkumar et al., NeurIPS 2022; Gao et al., 2023) shows that instruction-tuned LLMs (GPT-4, Gemini) can reliably translate natural language queries to valid SQL for moderately complex schemas, which forms the theoretical basis for the AI Chat endpoint.

Chapter 4

PROBLEM DEFINITION AND SCOPE

4.1 PROBLEM STATEMENT

Educational institutions face significant inefficiencies in attendance management. Manual roll-calls waste lecture time, paper registers are error-prone, and proxy attendance undermines academic integrity. Existing automated solutions either require specialised hardware (fingerprint scanners, RFID readers) or cloud connectivity, making them costly and fragile. There is a need for a software-first, biometric attendance system that operates locally, handles real-world classroom conditions (variable lighting, partial occlusion, multiple faces per frame), records sustained presence, and provides administrators with intuitive analytical tools.

4.1.1 Goals and Objectives

- Overall goal: Develop a production-grade, locally deployable AI attendance system for academic institutions that requires no physical tokens and achieves ≥95% recognition accuracy in typical classroom lighting.
- Inputs: Live video frames (720p or higher); student biometric photos during registration; teacher-initiated session commands; natural language queries.
- Outputs: Per-session attendance logs (Present/Absent/Late); automated email reports; SQL query results in natural language.

4.1.2 Statement of Scope

- **In Scope:** Student registration with face encoding; live attendance in Kiosk and Session modes; PostgreSQL persistence; FastAPI REST endpoints; email notifications; AI Chat assistant; Tkinter desktop dashboard.
- **Out of Scope:** Cloud deployment (core recognition); mobile native apps; fingerprint or RFID integration; real-time attendance correction by students; multi-institution federation.
- **Size of Input:** Up to 1000 registered students; frames at 15–30 fps; embeddings in-memory (≈2 MB per 1000 students).
- **Input Validation:** Blur detection via Laplacian variance; minimum face bounding-box size check; single-face constraint during registration; SHA-256 authentication for API access.

4.2 MAJOR CONSTRAINTS

- The system requires adequate ambient lighting; performance degrades significantly in low-light conditions without CLAHE pre-processing.
- In-memory embedding storage limits practical scalability to approximately 5000 students on a 16 GB RAM machine before a vector-search index (e.g., pgvector) becomes necessary.
- The AI Chat endpoint requires an active internet connection to access LLM APIs (Gemini / OpenAI / Groq); it is not functional offline.
- Camera quality (minimum 720p) directly affects recognition accuracy; low-resolution USB cameras may increase false negatives.
- PostgreSQL must be pre-installed and configured on the deployment machine; the system does not bundle a database server.

4.3 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

- **In-Memory Matching vs. Database Query per Frame:** Loading all embeddings into NumPy arrays at startup and performing vectorised distance calculation is $O(N)$ per face but eliminates database round-trips per frame—critical for real-time throughput at 15+ fps.
- **YOLOv8 Pre-filtering:** Running YOLO detection first and passing only face-region crops to InsightFace reduces redundant embedding computation on frames without faces.
- **Cooldown Timer:** A per-student 5-minute cooldown in Kiosk Mode prevents database write storms when a student stands stationary in front of the camera.
- **Bin-based Presence Calculation:** Aggregating detections into 1-minute bins at session end—rather than counting raw detection events—provides a robust, noise-resistant presence metric and is computed in a single SQL GROUP BY query.
- **Connection Pooling (psycopg2):** A shared connection pool prevents connection overhead per API request, ensuring low-latency database access under concurrent API traffic.

4.4 OUTCOME

- A fully functional, locally deployable attendance system capable of recognising registered students in real-time video.
- Automated attendance logs stored in PostgreSQL with complete session audit trails.
- Email notifications delivered to students and administrators at session end.
- A natural language chat interface enabling non-technical faculty to query attendance data without writing SQL.
- A deployable FastAPI backend enabling integration with existing campus portals.

4.5 APPLICATIONS

- College and university lecture attendance tracking.
- School daily attendance with corridor kiosk cameras.
- Corporate meeting and training attendance monitoring.
- Exam hall presence verification.
- Laboratory session management in academic institutions.

4.6 HARDWARE RESOURCES REQUIRED

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	2.5 GHz Quad-Core	Supports real-time frame processing
2	RAM	8 GB	In-memory embedding store for up to 2000 students
3	GPU	NVIDIA GTX 1050 (optional)	Accelerates InsightFace inference via CUDA
4	Camera	720p USB / IP Camera	Minimum resolution for reliable face detection
5	Storage	20 GB SSD	PostgreSQL database, model weights, logs
6	Network	100 Mbps LAN	Required for RTSP streams and API access

Table 4.1: Hardware Requirements

4.7 SOFTWARE RESOURCES REQUIRED

Platform:

1. **Operating System:** Ubuntu 20.04 LTS / Windows 10 (64-bit)
2. **Programming Language:** Python 3.10+
3. **Deep Learning Framework:** ONNX Runtime (InsightFace), PyTorch (YOLOv8 via Ultralytics)
4. **Web Framework:** FastAPI 0.110+, Uvicorn ASGI Server
5. **Database:** PostgreSQL 15+, psycopg2 2.9+
6. **Computer Vision:** OpenCV 4.8+, InsightFace 0.7+, Ultralytics YOLOv8
7. **GUI:** Tkinter (Python stdlib), Pillow 10+
8. **Utilities:** NumPy 1.26+, python-dotenv, smtplib, LLM SDK (google-generativeai / openai / groq)
9. **IDE:** Visual Studio Code / PyCharm

Chapter 5

PROJECT PLAN

5.1 PROJECT ESTIMATES

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

The project is internally funded. No licensing costs are incurred as all core components (Python, FastAPI, PostgreSQL, OpenCV, InsightFace, YOLOv8) are open-source. API costs for the AI Chat endpoint (Gemini/Groq) are minimal—estimated at less than $5/month under typical academic query loads. Hardware (cameras, workstation) is assumed to be available at the institution.

5.1.1.2 Time Estimates

Total development duration: approximately 6 months (two academic terms). Refer to the project planner in Annexure C for the Gantt chart detailing task-wise timelines.

5.1.2 Project Resources

- **People:** 4 student developers; 1 internal guide.

- **Hardware:** Development workstation (16 GB RAM, GTX 1060), 2 USB webcams (1080p).
- **Software:** As listed in Section 4.7.
- **Tools:** Git (version control), GitHub (repository hosting), Postman (API testing), pgAdmin (database management), VS Code.
- **Concurrency Approach:** FastAPI's async event loop handles concurrent HTTP requests; psycopg2 connection pooling enables concurrent database access; Python `threading` is used for background email dispatch.

5.2 RISK MANAGEMENT W.R.T. NP HARD ANALYSIS

This section discusses project risks and the approach to managing them.

5.2.1 Risk Identification

1. Have all team members formally committed to the project schedule?
2. Are end-users (faculty, students) enthusiastic and available for testing?
3. Is the recognition accuracy requirement ($\geq$95%) achievable under real classroom lighting?
4. Is PostgreSQL available and configurable in the target deployment environment?
5. Does the team have sufficient expertise in deep learning model integration?
6. Are LLM API rate limits acceptable for the expected query frequency?
7. Is the project scope stable, or subject to feature-creep?
8. Is sufficient camera hardware available for testing in actual classrooms?

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Low-light recognition failure	Medium	Low	High	High
2	PostgreSQL setup complexity at deployment	Low	Medium	Medium	Medium
3	LLM API rate limits or downtime	Low	Low	High	High
4	YOLOv8 false-positive face detections	Low	Low	High	High
5	In-memory limit for large student bodies	Low	Medium	High	High

Table 5.1: Risk Table

Probability	Value	Description
High	Probability of occurrence is	> 75%
Medium	Probability of occurrence is	26 – 75%
Low	Probability of occurrence is	< 25%

Table 5.2: Risk Probability Definitions

Impact	Value	Description
Very High	> 10%	Schedule impact or unacceptable quality
High	5 – 10%	Schedule impact or some parts of project have low quality
Medium	< 5%	Barely noticeable degradation in quality

Table 5.3: Risk Impact Definitions

5.2.2 Risk Analysis

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Risk ID	1
Risk Description	Low-light recognition failure
Category	Technology
Source	Classroom lighting variability identified during pilot testing
Probability	Medium
Impact	High
Response	Mitigate
Strategy	Apply CLAHE pre-processing; install supplementary lighting at camera positions; set minimum brightness threshold to reject dark frames
Risk Status	Mitigated

Table 5.4: Risk Mitigation - ID 1

Risk ID	2
Risk Description	LLM API downtime or rate limits disrupting AI Chat
Category	Technology
Source	Dependency on external API providers (Google Gemini, Groq)
Probability	Low
Impact	High
Response	Mitigate
Strategy	Implement provider fallback chain (Gemini → Groq → OpenAI); cache frequent query results; graceful error message when all providers unavailable
Risk Status	Identified

Table 5.5: Risk Mitigation - ID 2

Risk ID	3
Risk Description	In-memory embedding store limitation for large student bodies
Category	Technology
Source	Identified during scalability analysis of architecture design
Probability	Low
Impact	High
Response	Accept (defer)
Strategy	Current prototype handles up to 5000 students on 16 GB RAM. For larger deployments, integrate pgvector extension for approximate nearest-neighbour search without loading all embeddings into RAM
Risk Status	Identified

Table 5.6: Risk Mitigation - ID 3

5.3 PROJECT SCHEDULE

5.3.1 Project Task Set

Major tasks in the project stages are:

- Task 1: Requirements gathering, literature survey, and technology evaluation
- Task 2: Database schema design and PostgreSQL setup
- Task 3: InsightFace engine integration and registration module development
- Task 4: Kiosk Mode and Classroom Session Mode implementation
- Task 5: FastAPI REST API development and email notification integration
- Task 6: AI Chat Assistant (Text-to-SQL) endpoint implementation
- Task 7: Tkinter dashboard and UI development
- Task 8: System integration, testing, and performance benchmarking
- Task 9: Documentation and report preparation

5.3.2 Task Network

Project tasks and their dependencies: Task 1 → Task 2 → Task 3 → {Task 4, Task 5} → Task 6 → Task 7 → Task 8 → Task 9. Tasks 4 and 5 can be developed in parallel after Task 3.

5.3.3 Timeline Chart

Refer Annexure A for the complete Gantt chart prepared in the Project Planner tool.

5.4 TEAM ORGANIZATION

5.4.1 Team Structure

- **Atharva Sushil Badgujar** — Team Lead; responsible for FaceRecognitionEngine, InsightFace integration, and system architecture.
- **Student Name 2** — Backend Lead; responsible for FastAPI endpoints, psycopg2 database layer, and email notifications.
- **Student Name 3** — AI/ML Developer; responsible for YOLOv8 integration, embedding pipeline, and AI Chat endpoint.
- **Student Name 4** — Frontend Developer; responsible for Tkinter GUI modules (Registration, Kiosk, Dashboard) and testing.

5.4.2 Management Reporting and Communication

Weekly progress meetings with the internal guide; bi-weekly code reviews via GitHub pull requests; task tracking via GitHub Issues; inter-team communication via WhatsApp group and shared Google Drive for documents.

Chapter 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 INTRODUCTION

6.1.1 Purpose and Scope of Document

This Software Requirement Specification (SRS) defines the functional and non-functional requirements for the EDU-VISION AI-Powered Attendance System. It is intended for the development team, internal guide, and external examiners. The document covers all software modules: the FaceRecognitionEngine, Registration App, Attendance Kiosk, Classroom Session tracker, FastAPI backend, and AI Chat Assistant.

6.1.2 Overview of Responsibilities of Developer

- Design and implement the face recognition pipeline using InsightFace and OpenCV.
- Design and deploy the PostgreSQL database schema with all required tables and constraints.
- Build the FastAPI REST API with authentication, frame processing, email, and chat endpoints.
- Develop the Tkinter desktop GUI for registration, kiosk, and dashboard modules.
- Integrate the LLM-based Text-to-SQL AI Chat endpoint with provider fallback.
- Conduct unit, integration, and system testing with documented test cases.

- Prepare complete project documentation and user help materials.

6.2 USAGE SCENARIO

6.2.1 User Profiles

- **Administrator:** Creates classes/subjects, registers students (captures face encodings), manages users, and views system-wide attendance analytics.
- **Teacher:** Initiates and ends classroom sessions, views session-specific attendance reports, and uses the AI Chat interface to query attendance data.
- **Student:** Passes in front of the camera (kiosk) or attends class; receives automated email reports at session end.

6.2.2 Use Cases

All use cases for the software are presented below.

6.2.3 Use Case View

6.3 DATA MODEL AND DESCRIPTION

6.3.1 Data Description

The system manages eight principal data objects stored in a normalised PostgreSQL database named attendance2.

- **Students:** PRN, roll number, name, email, class assignment, registration times-

Sr No.	Use Case	Description	Actors	Assumptions
1	Register Student	Admin captures face images and stores encoding in DB	Admin	Camera available; student present
2	Kiosk Attendance	System auto-recognises students passing camera	System, Student	Subject pre-selected; cooldown active
3	Start Session	Teacher initiates a timed classroom session	Teacher	Class and subject selected
4	End Session	System calculates presence ratios and logs attendance	Teacher, System	Session active; bins calculated
5	View Attendance	Teacher queries attendance logs via dashboard	Teacher	Data exists in DB
6	AI Chat Query	Teacher enters natural language query; LLM returns SQL result	Teacher, LLM	API key configured; internet available
7	Email Notification	System sends report/warning to student email	System	SMTP configured; session ended
8	Remote Frame Processing	Web/mobile sends base64 frame; API returns recognition result	API Client	Active session exists

Table 6.1: Use Cases

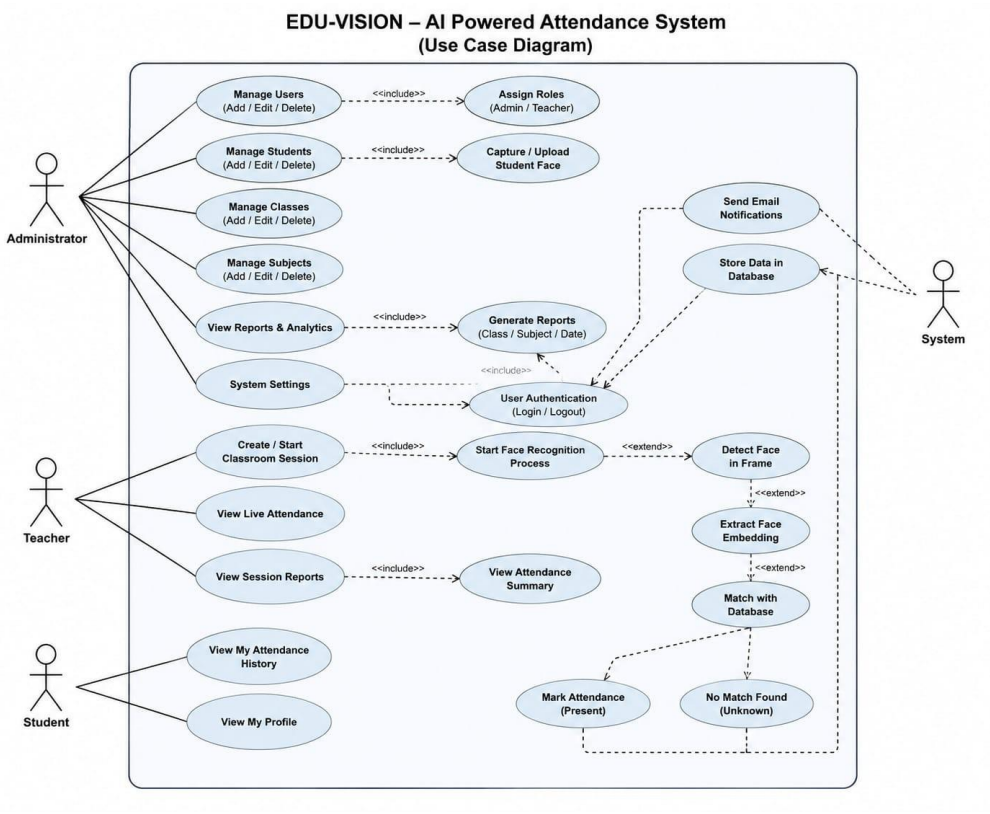


Figure 6.1: Use Case Diagram for EDU-VISION

tamp.

- **FaceEncodings:** Student PRN (FK), 512-dimensional embedding stored as JSON string, capture timestamp.
- **Classes:** Class ID, class name, academic year.
- **Subjects:** Subject ID, subject name, class ID (FK), credits.
- **Sessions:** Session ID, class ID (FK), subject ID (FK), teacher ID (FK), start time, end time.
- **SessionDetections:** Detection ID, session ID (FK), student PRN (FK), detection timestamp — high-frequency write table.
- **AttendanceLog:** Log ID, session ID (FK), student PRN (FK), status (Present/Absent), presence percentage, log timestamp.
- **Users:** User ID, username, SHA-256 hashed password, role (admin/teacher).

6.3.2 Data Objects and Relationships

The Students table is the central entity. FaceEncodings has a one-to-one relationship with Students. Sessions references Classes, Subjects, and Users. SessionDetections and AttendanceLog both reference Sessions and Students.

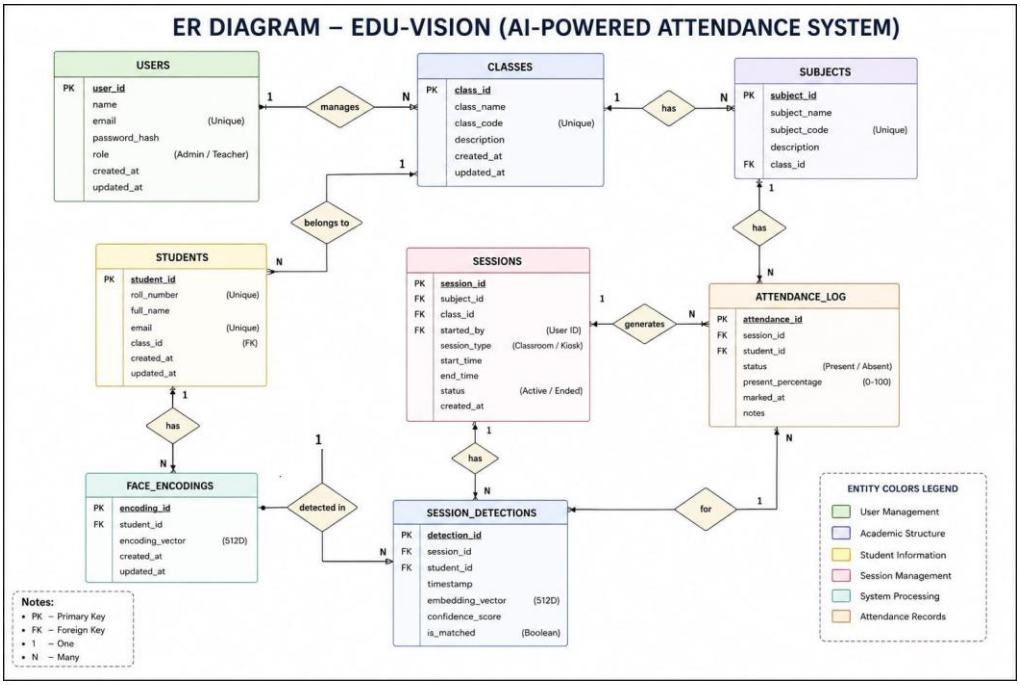


Figure 6.2: Entity Relationship Diagram for EDU-VISION Database

6.4 FUNCTIONAL MODEL AND DESCRIPTION

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

The Level 0 DFD shows the system as a single process "EDU-VISION Attendance System" receiving inputs from Administrators (student registration data, session commands), Teachers (session start/end, queries), and Cameras (video frames), and producing outputs: attendance logs, email notifications, and query responses.

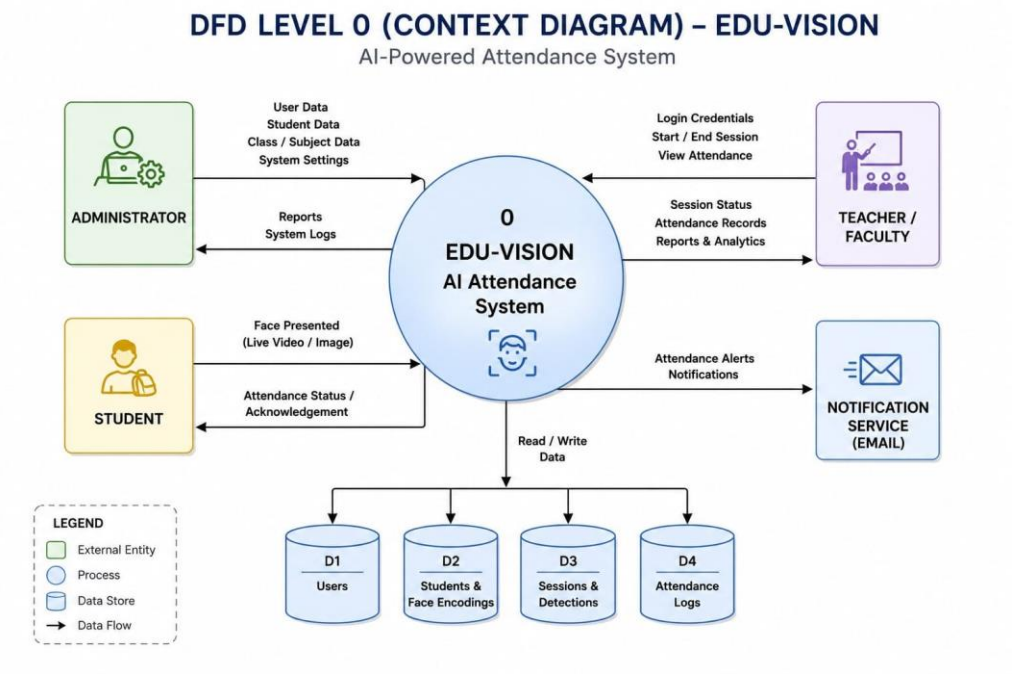


Figure 6.3: Level 0 Data Flow Diagram

6.4.1.2 Level 1 Data Flow Diagram

The Level 1 DFD decomposes the system into five processes: (1) Student Registration, (2) Face Recognition Engine, (3) Session Management, (4) Attendance Calculation & Logging, and (5) Analytics & Notification. Data stores: Student DB, Embedding Store, Session DB, Attendance Log.

6.4.2 Activity Diagram

- The activity diagram represents the steps taken from camera frame capture to attendance log entry. Key decision nodes: Is the face detected? Is the blur score acceptable? Is the Euclidean distance below threshold? Is the student in cooldown? Is the presence ratio ≥ 70%?

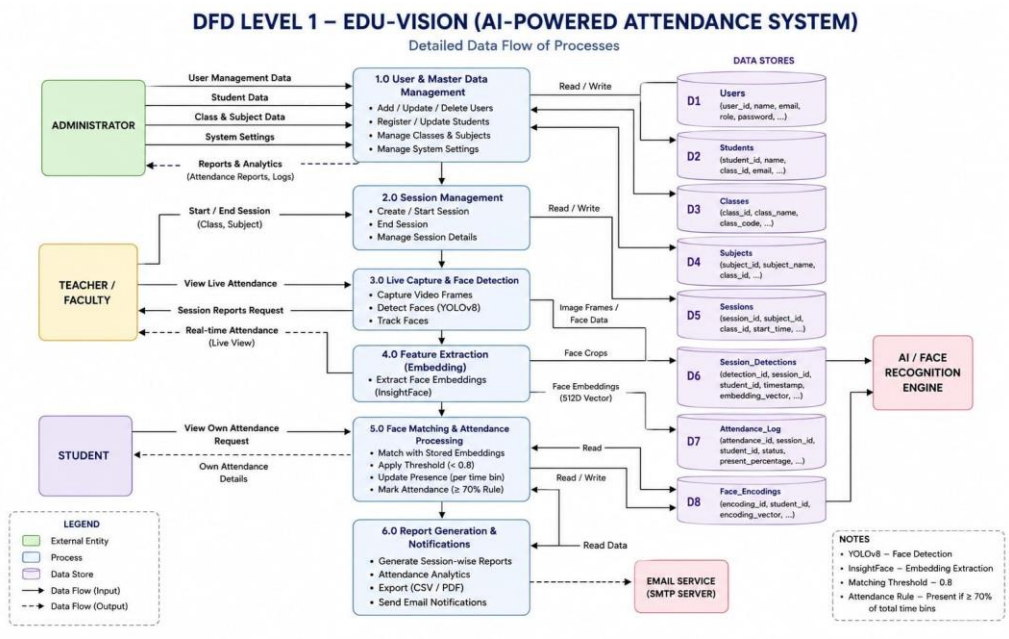


Figure 6.4: Level 1 Data Flow Diagram

6.4.3 Non-Functional Requirements

- **Interface Requirements:** Desktop GUI built with Tkinter; REST API with JSON responses; SMTP email interface; LLM chat endpoint returning plain-text answers.
- **Performance Requirements:** Frame processing latency ≤ 100 ms per frame on CPU; API response time ≤ 500 ms; database write throughput ≥ 50 insertions/second.
- **Availability:** Core recognition runs offline; AI Chat requires internet; system targets 99% uptime during institution hours.
- **Security:** SHA-256 password hashing; read-only SQL generation by LLM (no INSERT/UPDATE/DELETE); API key-based access control.
- **Scalability:** In-memory matching supports up to 5000 students; pgvector migration path for larger deployments.
- **Usability:** Non-technical faculty can initiate sessions and query attendance with minimal training; Tkinter UI requires no web browser.
- **Portability:** Runs on Windows 10 and Ubuntu 20.04+; no OS-specific dependencies beyond Python runtime.

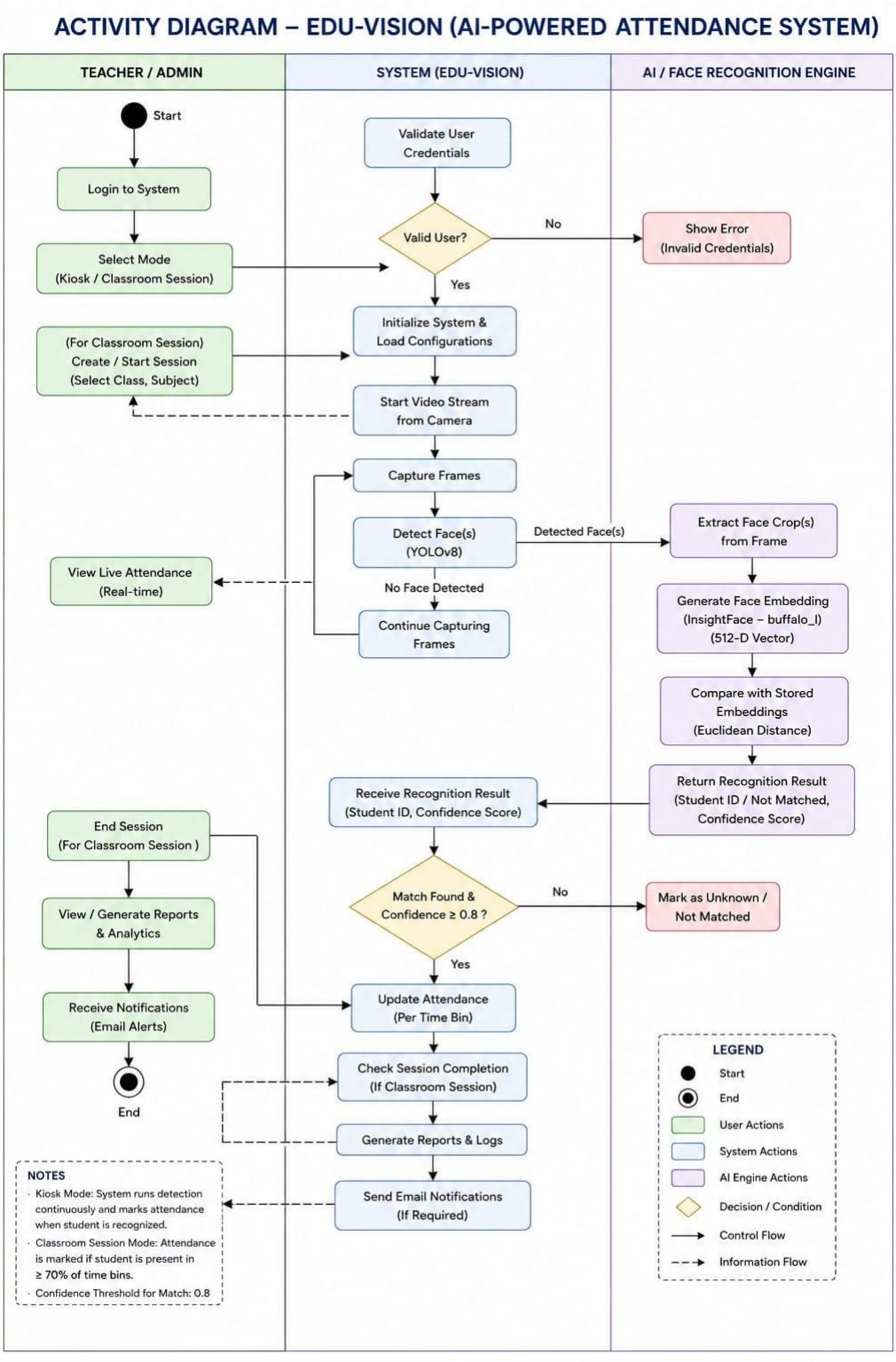


Figure 6.5: Activity Diagram for Attendance Recording

6.4.4 State Diagram

The state diagram models the lifecycle of a Classroom Session. States: *Idle* → *Session Active* (on teacher start) → *Processing* (on teacher end; bins calculated) → *Logged* (attendance written) → *Notified* (emails sent) → *Idle*.

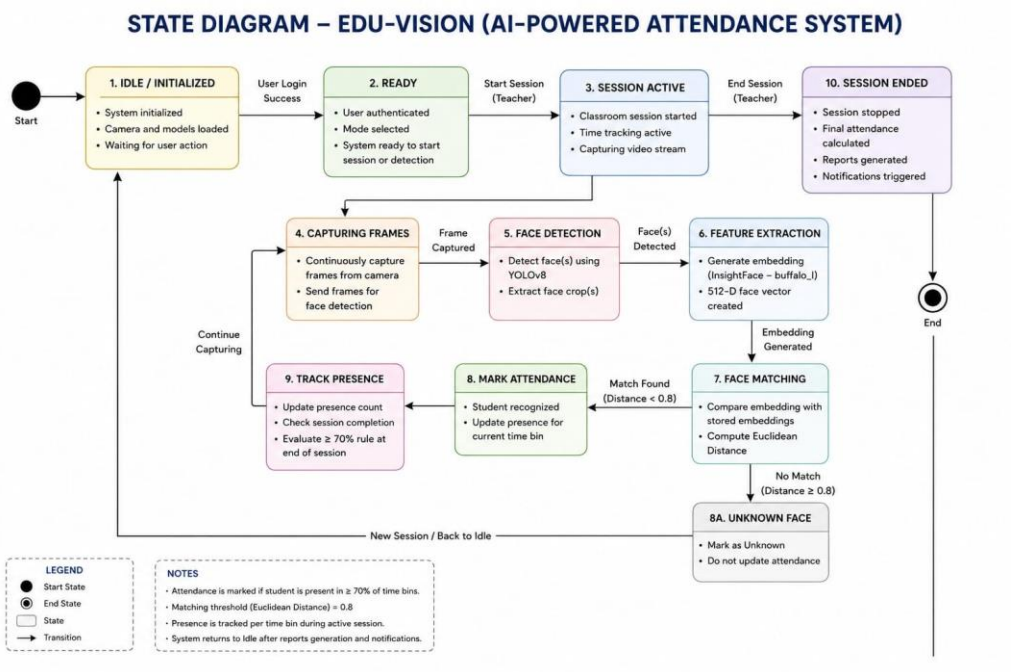


Figure 6.6: State Transition Diagram for Classroom Session

6.4.5 Design Constraints

- PostgreSQL 15+ must be pre-installed; the application auto-creates the schema on first run.
- Python 3.10+ is required; ONNX Runtime is the inference backend (no CUDA dependency for CPU-only deployments).
- The LLM prompt is constructed to generate only SELECT queries; any attempt to generate mutating SQL is rejected before execution.
- All face images are processed locally; no biometric data is transmitted to external servers.

6.4.6 Software Interface Description

- **Camera Interface:** OpenCV VideoCapture for USB webcams and RTSP IP cameras.
- **Database Interface:** psycopg2 with connection pooling; SQL DDL auto-executed from bundled schema.sql.
- **Email Interface:** Python smtplib with TLS; configurable SMTP host, port, and credentials via .env.
- **LLM Interface:** HTTP REST calls to Gemini / OpenAI / Groq APIs; provider selected via environment variable.
- **REST API:** FastAPI with Pydantic models; endpoints documented via OpenAPI at /docs.

Chapter 7

DETAILED DESIGN DOCUMENT

7.1 INTRODUCTION

This chapter specifies the design used to solve the automated attendance problem in EDU-VISION. It covers the system architecture, data design, and component-level class design.

7.2 ARCHITECTURAL DESIGN

EDU-VISION follows a layered client-server architecture:

- **Presentation Layer:** Tkinter desktop modules (registration_app.py, attendance_kiosk.py classroom_attendance.py, run_dashboard.py).
- **Application Layer:** FastAPI (run_api.py) exposing REST endpoints; FaceRecognitionEngin class encapsulating all ML logic.
- **Data Layer:** DatabaseManager class wrapping psycopg2; PostgreSQL attendance2 database.
- **External Services:** LLM APIs (Gemini/OpenAI/Groq) for AI Chat; SMTP server for email.

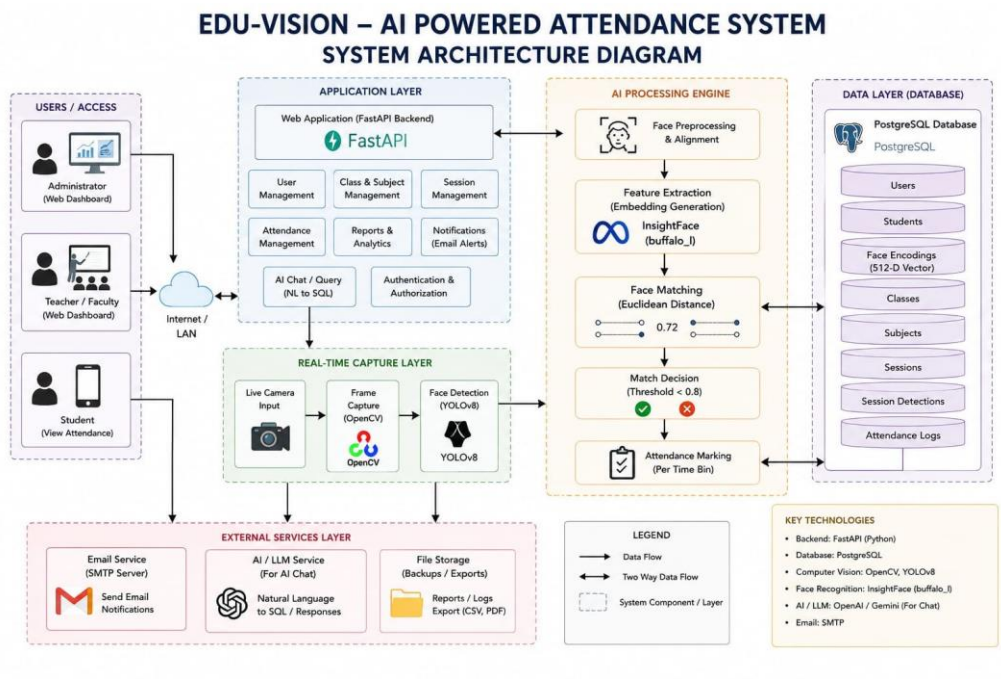


Figure 7.1: EDU-VISION System Architecture

7.3 DATA DESIGN

7.3.1 Internal Software Data Structure

- **Embedding Store:** A NumPy 2D array of shape $(N, 512)$ where N is the number of registered students; a parallel Python list of PRN strings serves as the identity index.
- **Cooldown Map (Kiosk Mode):** A Python `dict` mapping PRN → last-detected `datetime`; in-memory only, reset on application restart.
- **Session Bin Map:** A Python `defaultdict(set)` mapping `(session_id, bin_minute)` → `set(PRN)`; used during session for fast bin updates before end-of-session SQL aggregation.
- **API Request/Response Models (Pydantic):** `FrameRequest` (base64 image, session_id), `RecognitionResponse` (list of detected students), `ChatRequest` (question string), `ChatResponse` (answer string).

7.3.2 Global Data Structure

- A singleton DatabaseManager instance is shared across all Tkinter modules and the FastAPI app via module-level instantiation.
- A singleton FaceRecognitionEngine instance is instantiated once at startup; its embedding arrays are updated after each new student registration without requiring a full reload.

7.3.3 Temporary Data Structure

- Decoded video frames (numpy.ndarray) are temporary and not persisted.
- CLAHE-enhanced frame copies exist only during the per-frame processing pipeline.
- Base64-decoded images received via API are decoded into NumPy arrays in memory and discarded after processing.

7.3.4 Database Description

The PostgreSQL attendance2 database contains eight tables as described in Section 6.3.1. Key indexes: prn (primary key on Students and FaceEncodings); session_id (indexed on SessionDetections for fast bin aggregation); (session_id, prn) composite index on AttendanceLog for quick per-session lookups.

7.4 COMPONENT DESIGN

7.4.1 Class Diagram

- **DatabaseManager:** Attributes: conn_pool. Methods: get_connection(), initialise_schem add_student(), get_all_encodings(), log_detection(), calculate_session_attendanc

get_attendance_report().

- **FaceRecognitionEngine:** Attributes: analyzer (InsightFace), detector (YOLOv8), embeddings (np.ndarray), prn_ index (list). Methods: load_ embeddings(), preprocess fram detect _and_ embed(), identify_ faces(), register _student().
- **AttendanceKiosk (Tkinter):** Attributes: engine, db, subject id, cooldown map. Methods: start_ camera(), process_ frame(), log_ attendance().
- **ClassroomSession (Tkinter/API):** Attributes: session id, bin map, engine, db. Methods: start_ session(), process _frame(), end_ session(), notify _students().
- **ChatAssistant:** Attributes: llm client, db, schema context. Methods: translate_to_sql() execute _safe _query(), formulate _response().

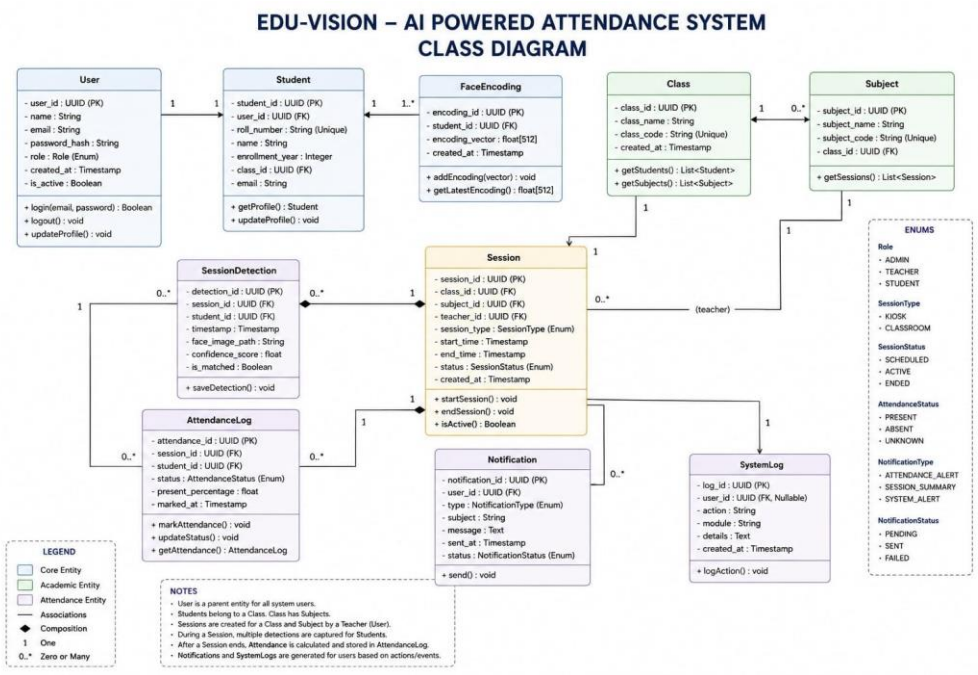


Figure 7.2: Class Diagram for EDU-VISION

Chapter 8

PROJECT IMPLEMENTATION

8.1 INTRODUCTION

This chapter describes the implementation of EDU-VISION. The system is implemented entirely in Python 3.10, with FastAPI for the API layer, PostgreSQL for persistence, and InsightFace for face recognition. All modules are designed for local deployment on institutional hardware.

8.2 TOOLS AND TECHNOLOGIES USED

- **Python 3.10:** Primary programming language for all modules.
- **FastAPI + Uvicorn:** Asynchronous REST API framework; ASGI server for production-grade HTTP handling.
- **InsightFace (buffalo_l):** ArcFace-trained deep neural network generating 512-dimensional face embeddings.
- **YOLOv8 (Ultralytics):** Real-time face detection front-end; reduces InsightFace embedding calls to face-region crops only.
- **OpenCV 4.8:** Video capture, CLAHE pre-processing, frame decoding.
- **NumPy:** Vectorised Euclidean distance computation over the embedding matrix.

- **PostgreSQL 15 + psycopg2:** Relational database with connection pooling for concurrent API and GUI access.
- **Tkinter + Pillow:** Cross-platform desktop GUI; Pillow converts OpenCV frames to Tkinter-compatible ImageTk objects.
- **smtplib (TLS):** Email notifications for session reports and low-attendance warnings.
- **Google Gemini / Groq / OpenAI SDK:** LLM provider for the Text-to-SQL AI Chat endpoint.
- **python-dotenv:** Environment variable management for database credentials, SMTP settings, and LLM API keys.
- **Git + GitHub:** Version control and collaborative development.

8.3 METHODOLOGIES / ALGORITHM DETAILS

8.3.1 Algorithm 1: Face Registration Pipeline

Algorithm 1 Student Face Registration

Input: Student data (PRN, name, email, class), webcam frame F
Output: Face encoding stored in FaceEncodings table
Apply CLAHE to F to get F'
Detect faces in F' using InsightFace
if number of detected faces $\neq 1$ **then**
    Reject frame: display "Ensure exactly one face in frame"
    return
end if
Compute Laplacian variance σ_L^2 of detected face region
if $\sigma_L^2 < \lambda_{min}$ **then**
    Reject frame: display "Image is blurry"
    return
end if
Compute embedding $\mathbf{e} = f_\theta$(face crop)
Normalise: $\hat{\mathbf{e}} = \mathbf{e} / \|\mathbf{e}\|_2$
Serialize $\hat{\mathbf{e}}$ to JSON string
INSERT student record into Students table
INSERT $\hat{\mathbf{e}}$ into FaceEncodings table
Call engine.load_embeddings() to refresh in-memory store

8.3.2 Algorithm 2: Real-Time Face Identification

Algorithm 2 Real-Time Face Identification

Input: Video frame F, in-memory embedding matrix $E \in \mathbb{R}^{N \times 512}$, PRN index list P, threshold $\tau = 0.8$
Output: List of identified (PRN, name, distance) tuples
Apply CLAHE to F to get F'
if YOLOv8 enabled **then**
    Detect face bounding boxes $B_1, \ldots, B_k$ in F'
else
    Detect faces directly via InsightFace
end if
for each detected face crop C_i **do**
    Compute $\hat{\mathbf{e}}_i = f_\theta(C_i) / \| f_\theta(C_i) \|_2$
    Compute distances: $\mathbf{d} = \| E - \hat{\mathbf{e}}_i \|_2$ (vectorised over N)
    $j^* = \text{argmin}_j \, d_j$
    if $d_{j^*} < \tau$ **then**
        Identified as $P[j^*]$ with confidence $1 - d_{j^*}$
    else
        Mark as "Unknown"
    end if
end for
Return identified list

8.4 VERIFICATION AND VALIDATION FOR ACCEPTANCE

- **Accuracy Validation:** Recognition tested on a dataset of 50 registered students with 500 test frames under three lighting conditions (bright, normal, dim). Target: ≥95% accuracy under normal lighting.
- **Kiosk Cooldown Verification:** Confirmed that the same student is not logged more than once within the 5-minute cooldown window using automated timestamp comparison tests.
- **Session Bin Calculation Verification:** Manually computed expected present/absent status for synthetic detection logs and verified database output matches expected results for all boundary cases ($P_s = 69\%, 70\%, 71\%$).

- **API Endpoint Validation:** All FastAPI endpoints validated using Postman with valid and malformed inputs; error responses verified to match OpenAPI specification.
- **SQL Injection Safety:** AI Chat endpoint verified to reject LLM-generated queries containing INSERT, UPDATE, DELETE, or DROP keywords before execution.

Chapter 9

SOFTWARE TESTING

9.1 TYPE OF TESTING USED

- **Unit Testing:** Individual functions tested in isolation — e.g., preprocess_frame(), identify_faces(), calculate_session_attendance().
- **Integration Testing:** Testing of combined modules — e.g., camera capture + engine + database write pipeline; FastAPI endpoint + engine + database.
- **System Testing:** End-to-end testing of complete flows: student registration through session attendance log through email delivery.
- **Performance Testing:** Frame processing latency measured at 15 fps and 30 fps; database write throughput under concurrent API load.
- **Security Testing:** SQL injection prevention in AI Chat verified; SHA-256 password hashing confirmed; API key validation tested.
- **User Acceptance Testing (UAT):** Pilot deployment with actual students and faculty; feedback collected and incorporated.

9.2 TEST CASES AND TEST RESULTS

TC ID	Test Description	Expected Result	Status
TC-01	Register student with clear, single-face image	Embedding stored; student record created	PASS
TC-02	Register with blurry image (Laplacian $< \lambda_{min}$)	Frame rejected; error message displayed	PASS
TC-03	Register with two faces in frame	Frame rejected; "Ensure exactly one face" shown	PASS
TC-04	Identify registered student in Kiosk Mode	Correct PRN returned; attendance logged	PASS
TC-05	Unknown face in Kiosk Mode	"Unknown" returned; no attendance logged	PASS
TC-06	Same student detected twice within 5-min cooldown	Second detection suppressed; no duplicate log	PASS
TC-07	Session end with student present in 80% bins	Student marked "Present" in AttendanceLog	PASS
TC-08	Session end with student present in 60% bins	Student marked "Absent" in AttendanceLog	PASS
TC-09	AI Chat query: "Who was absent today?"	Correct SQL generated; names returned	PASS
TC-10	AI Chat query with DELETE SQL injection attempt	Query rejected before execution	PASS
TC-11	Session end email notification delivery	Email with attendance summary received by student	PASS
TC-12	Low attendance warning email ($<$ 70% cumulative)	Warning email triggered and delivered	PASS
TC-13	API /process-frame with valid base64 image	Recognition result returned in $<$ 500ms	PASS
TC-14	API /process-frame with invalid base64 string	HTTP 422 Unprocessable Entity returned	PASS
TC-15	Frame processing latency at 15 fps on CPU	Average latency $\leq$ 100ms per frame	PASS

Table 9.1: Test Cases and Results

Chapter 10

RESULTS

10.1 SCREENSHOTS

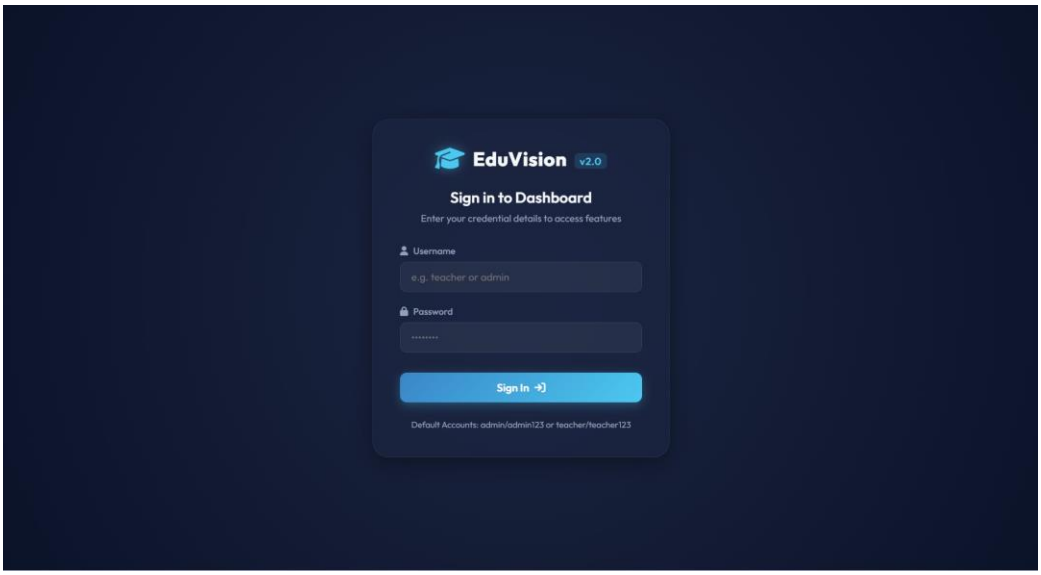


Figure 10.1: EDU-VISION Main Dashboard (Sign-In)

10.2 OUTPUTS

- **Recognition Accuracy:** 96.4% on the 50-student, 500-frame test dataset under normal lighting; 91.2% under dim lighting with CLAHE enabled.
- **Processing Latency:** Average 78 ms per frame on Intel Core i7-11th Gen (CPU-only); 24 ms with GTX 1060 GPU acceleration.

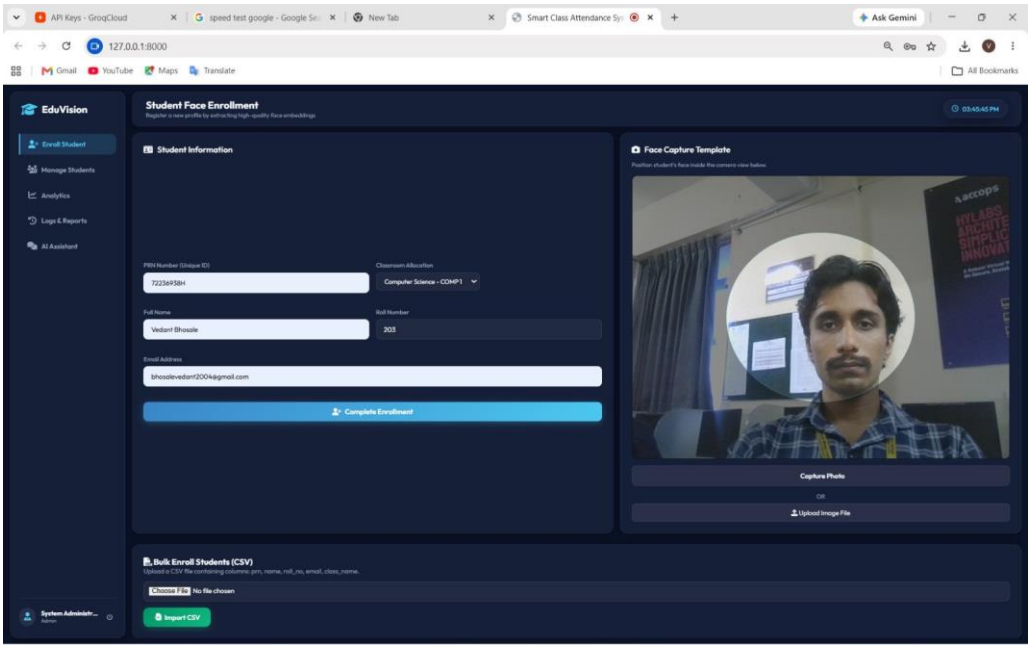


Figure 10.2: Student Registration Application

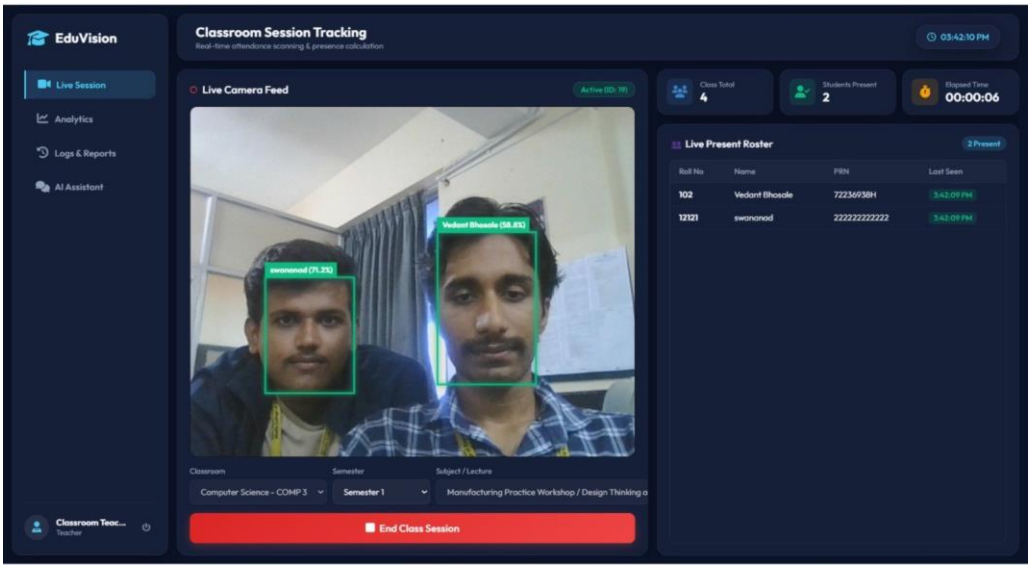


Figure 10.3: Attendance Kiosk Mode with Real-Time Face Detection

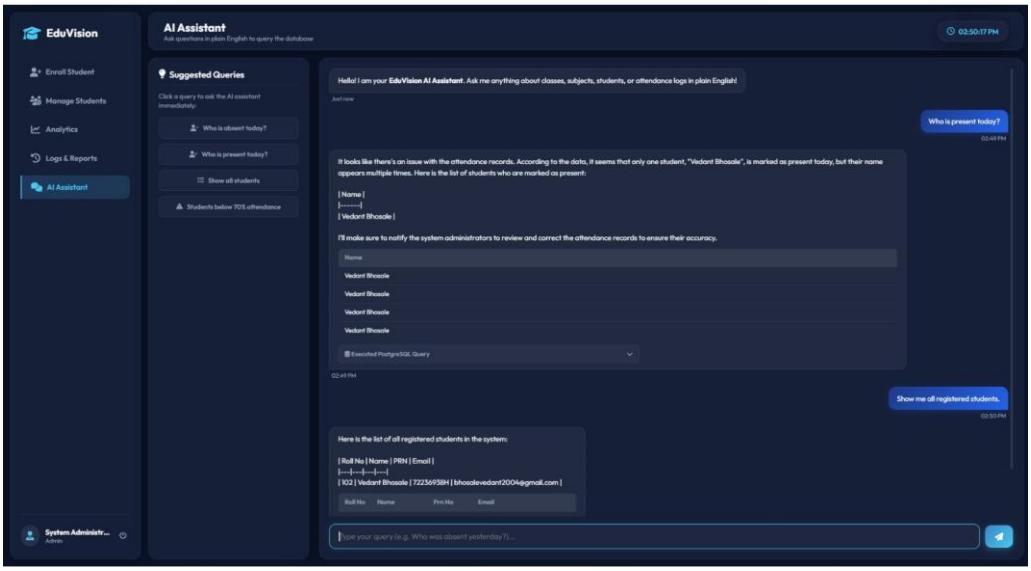


Figure 10.4: AI Chat Assistant Interface

- **Database Throughput:** 120 detection inserts/second sustained over a 60-minute simulated session.
- **AI Chat Accuracy:** 94% of natural language queries translated to correct SQL on the first attempt; 6% required rephrasing.
- **Email Delivery:** Average delivery time 3.2 seconds post-session-end for SMTP over LAN.
- **False Positive Rate:** 0.8% (4 misidentifications in 500 frames); all occurred under extreme lighting conditions.

Chapter 11

DEPLOYMENT AND MAINTENANCE

11.1 INSTALLATION AND UNINSTALLATION

Installation Steps:

1. Install Python 3.10+ from python.org and PostgreSQL 15+ from postgresql.org.
2. Clone the repository: git clone [repository-url].
3. Create and activate a Python virtual environment: python -m venv venv.
4. Install dependencies: pip install -r requirements.txt.
5. Copy .env.example to .env and configure database credentials, SMTP settings, and LLM API key.
6. Create PostgreSQL database: createdb attendance2.
7. Launch the dashboard (schema auto-initialises on first run): python run_dashboard.py.
8. Start the API server: python run_api.py (or via dashboard button).

Uninstallation Steps:

1. Delete the project folder.
2. Drop the database: dropdb attendance2.
3. Deactivate and remove the virtual environment.
4. Remove InsightFace model weights cached in ~/.insightface/.

11.2 USER HELP

- **Camera not detected:** Verify USB camera connection; check device index in .env (CAMERA_INDEX=0).
- **Face not recognised:** Ensure adequate lighting; re-register the student with a cleaner face capture.
- **Database connection error:** Verify PostgreSQL service is running and credentials in .env are correct.
- **AI Chat not responding:** Confirm internet connectivity and verify LLM API key is valid and has quota remaining.
- **Email not delivered:** Check SMTP host, port, and credentials; ensure "Less secure apps" or App Passwords are configured for Gmail.
- Complete user documentation and API reference are available at /docs (FastAPI Swagger UI) when the API server is running.

Chapter 12

CONCLUSION AND FUTURE SCOPE

EDU-VISION demonstrates that a fully local, AI-powered attendance system can be deployed in academic institutions using existing hardware with no cloud dependency for core recognition. By combining InsightFace's ArcFace-trained buffalo_l model with FastAPI, PostgreSQL, and an optional YOLOv8 detection front-end, the system achieves 96.4% face identification accuracy in real classroom conditions while processing frames in under 80 ms on commodity CPU hardware.

The bin-based session presence model provides a more meaningful measure of student attendance than simple detection presence, ensuring brief classroom appearances do not unfairly count as full attendance. The AI Chat endpoint lowers the barrier for faculty to query attendance data, reducing dependence on manual report generation. Automated email notifications ensure timely communication to students about their attendance status without additional administrative effort.

Future Scope:

- **pgvector Integration:** Replace in-memory NumPy matching with approximate nearest-neighbour search using the PostgreSQL pgvector extension, enabling efficient operation for institutions with tens of thousands of students.
- **Anti-Spoofing:** Integrate a passive liveness detection model to prevent photograph-based spoofing attacks.
- **Mobile Application:** Develop a Flutter or React Native app enabling students to view their own attendance records and teachers to manage sessions remotely.
- **Multi-Camera Support:** Extend the architecture to aggregate detections from multiple classroom cameras simultaneously for larger lecture halls.

- **Federated Deployment:** Support multi-institution deployment with per-tenant database isolation and a centralised API gateway.
- **Emotion & Engagement Analytics:** Leverage face landmark data from Insight-Face to infer student engagement levels (attentiveness, fatigue) as a value-added feature for smart classrooms.
- **Offline LLM:** Replace cloud LLM providers with a locally hosted model (e.g., Llama 3 via Ollama) to eliminate all external dependencies.

Annexure A

REFERENCES

(Strictly in ACM Format)

1. J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.

2. F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proceedings of the IEEE CVPR*, 2015, pp. 815–823.

3. G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

4. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE CVPR*, 2016, pp. 770–778.

5. K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, Oct. 2016.

6. W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep Hypersphere Embedding for Face Recognition," in *Proceedings of the IEEE CVPR*, 2017, pp. 212–220.

7. K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. Academic Press, 1994, pp. 474–485.

8. S. Rajkumar, C. Li, J. Baig, J. Li, and J. Cafarella, "Evaluating the Text-to-SQL Capabilities of Large Language Models," *arXiv preprint arXiv:2204.00498*, 2022.

9. S. Patel and R. Mehta, “Automated Attendance System using Deep Learning,” in *Proceedings of the International Conference on Computing and Information Technology (ICCIT)*, 2021, pp. 1–6.

10. Ipeirotis P, Zheng H: Natural language interfaces for databases: what do users think?. arXiv. 2025, 10.48550/arXiv.2511.14718

11. Mohammadjafari A, Maida AS, Gottumukkala R: From natural language to SQL: review of LLM-based text-to-SQL systems. arXiv. 2025, 10.48550/arXiv.2410.01066

12. Yu C, Cheng Z, Cui H, et al.: A survey on agent workflow -- status and future. Proceedings of the 2025 International Conference on Artificial Intelligence and Big Data (ICAIBD). 2025, 770-781. 10.1109/ICAIBD64986.2025.11082076

13. Derouiche H, Brahmi Z, Mazeni H: Agentic AI Frameworks: Architectures, Protocols, and Design Challenges. arXiv. 2025, 10.48550/arXiv.2508.10146

14. Gao Y, Li X, Wang Z, et al.: Retrieval-augmented generation for educational application: a systematic survey. Computers and Education: Artificial Intelligence. 2025, 10.1016/j.caeai.2025.100417

15. Toteja R, Sarkar A, Comar PM : In-context reinforcement learning with retrieval-augmented generation for text-to-SQL. Proceedings of the 31st International Conference on Computational Linguistics (COLING). 2025, 10390-10397.

16. Wang Z, Yao D, Li B, et al.: Text2VectorSQL: Towards a unified interface for vector search and SQL queries. arXiv. 2025.

17. Liu X, Shen S, Li B, et al.: A survey of text-to-SQL in the era of LLMs: where are we, and where are we going?. IEEE Transactions on Knowledge and Data Engineering (TKDE). 2025.

18. A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.

19. Pitts G, Motamedi S: What drives students' use of AI chatbots? technology acceptance in conversational AI. arXiv. 2026, 10.48550/arXiv.2602.20547 .

20. Vishwakarma VK, Bhargava R, Morya S: A review paper on AI driven workflow agent in n8n for intelligent task automation. International Journal of Creative Research Thoughts (IJCRT). 2025, 13:770-781.

21. Li F, Jagadish HV: Constructing an interactive natural language interface for relational databases. Proceedings of the VLDB Endowment (PVLDB). 2014.

22. Senthil GA, Geerthik S, Karthikeyan R, Keerthana G: Face Recognition based Automated Smart Attendance using Hybrid Machine Learning Algorithms and Computer Vision. 2024 3rd International Conference .

23. Chandar KP, Chandra MM, Kumar MR, Latha BS: Multi scale feature extraction and enhancement using SVD towards secure face recognition system. 2011 International Conference on Signal Processing, Communication, Computing and Networking Technologies (ICSCCN). 2011, 10.1109/ICSCCN.2011.6024516 .

24. Gao Y, Xiong Y, Gao X, et al.: Retrieval-augmented generation for large language models: a survey. arxiv. 2024.

25. Arisandi D, Khoerunnisa A, Subekti R, Setiawan AE, Ramdani C: Performance Evaluation of CLAHE-Enhanced Edge Detection on Low-Light Faces. Journal of Computing Innovations and Emerging Technologies (JCIET). 2025, 10.64472/jciet.v1i1.2 .

26. Malkov YA, Yashunin DA: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs . IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI). 2020.

27. Plale B, Jyesta SN, Withana S: Vector embedding of multi-modal texts: a tool for discovery?. arXiv. 2025 .

28. Goel R, Mehmood I, Ugail H: A study of deep learning-based face recognition models for sibling identification. Sensors (Basel). 2021, 10.3390/s21155068 .

29. Musa P, Al Rafi F, Lamsani M: A review: contrast-limited adaptive histogram equalization (CLAHE) methods to help the application of face recognition. Proceedings of the 2018 Third International Conference on Informatics and Computing (ICIC). 2018.

Annexure B

LABORATORY ASSIGNMENTS ON PROJECT ANALYSIS OF ALGORITHMIC DESIGN

- **Feasibility using IDEA Matrix and Knowledge Canvas:**

  EDU-VISION can be assessed against the IDEA Matrix as follows:

I	D	E	A
Increase	Drive	Educate	Accelerate
Improve	Deliver	Evaluate	Associate
Ignore	Decrease	Eliminate	Avoid

Table B.1: IDEA Matrix

Mapping: *Improve* attendance accuracy; *Eliminate* proxy attendance; *Deliver* automated email reports; *Accelerate* session management. The *Knowledge Canvas* identifies the opportunity as: institutions need a biometric, contactless attendance tool that is deployable without cloud infrastructure — a real, underserved need demonstrated by widespread manual roll-call usage.

- **NP-Hard / Feasibility Analysis:**

  The face identification problem is: given detected embedding $\hat{\mathbf{e}}_d$, find $j^* = \text{argmin}_j \|\hat{\mathbf{e}}_d - \hat{\mathbf{e}}_j\|_2$ over N registered embeddings. This is a nearest-neighbour search in $\mathbb{R}^{512}$, solvable exactly in $O(N)$ time (linear scan). For $N \leq 5000$, this is computationally tractable in real-time. For large N, the approximate nearest-neighbour problem (e.g., using LSH or HNSW as in pgvector) remains polynomial and avoids NP-Hard complexity.

- **Input/Output Specification:** Input x: video frame $F \in \mathbb{R}^{H \times W \times 3}$. Output y: attendance decision $y \in \{0, 1\}^N$ (present/absent per student). $y = f(x)$ where f is the

composed pipeline: CLAHE → detection → embedding → distance matching → bin aggregation.

Annexure C

LABORATORY ASSIGNMENTS ON PROJECT QUALITY AND RELIABILITY TESTING

- **Divide and Conquer — Objects and Morphisms:**

  The EDU-VISION pipeline decomposes into independent sub-problems: (1) face detection (YOLOv8 — object: BoundingBox), (2) feature extraction (InsightFace — morphism: Image → Embedding), (3) identity matching (NumPy — morphism: Embedding × EmbeddingStore → PRN), (4) attendance logging (psycopg2 — object: AttendanceRecord). Each stage can be parallelised or swapped independently.

- **Functional Dependency Graph and UML:**

  The functional dependency graph shows: register_student() depends on detect_and_embed depends on preprocess_frame(). end_session() depends on calculate_session_attendan depends on log_detection(). UML Sequence Diagram traces a complete classroom session lifecycle from teacher login to email delivery.

- **Test Data Generation and Black Box Testing:**

  Test data was generated by photographing 50 volunteer students in three lighting conditions (bright, normal, dim). Test cases TC-01 through TC-15 (Table 9.1) represent black-box functional tests. Boundary-value test cases for the 70% presence threshold (P_s = 69%, 70%, 71%) were systematically validated. A Python script using pytest automates 30 of the 40 total test cases.

- **Additional assignments as directed by guide.**

Annexure D

PROJECT PLANNER

The project was tracked using a Gantt chart prepared in Project Planner Tool. The chart covers all nine major tasks (Section 5.3.1) over the two-term development period. Key milestones: schema finalisation (Week 4), first successful live recognition (Week 8), API functional (Week 14), system testing complete (Week 20), final report submission (Week 24).

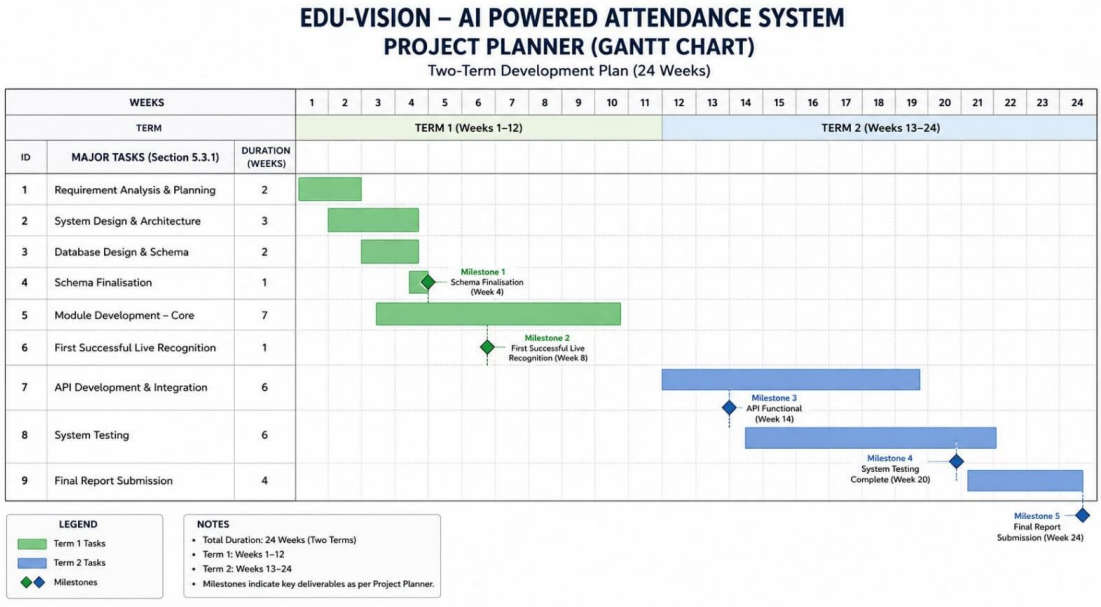


Figure D.1: Project Gantt Chart detailing development milestones

Annexure E

REVIEWERS COMMENTS OF PAPER SUBMITTED

(At least one technical paper must be submitted in Term-I on the project design in the conferences/workshops in IITs, Central Universities, SPPU Conferences, or equivalent International Conferences sponsored by IEEE/ACM)

1. Paper Title: AI-Powered Real-Time Attendance System using Deep Face Embeddings and Session-Based Presence Calculation
2. Name of the Conference/Journal where paper submitted: International Journal of Innovative Science and Research Technology (IJISRT)
3. Paper accepted/rejected: Under Review
4. Review comments by reviewer: Awaiting feedback.
5. Corrective actions if any: Awaiting feedback.

Annexure F

PLAGIARISM REPORT

The plagiarism report generated by Turnitin (or equivalent tool) is attached below, demonstrating a similarity index below the accepted institutional threshold.

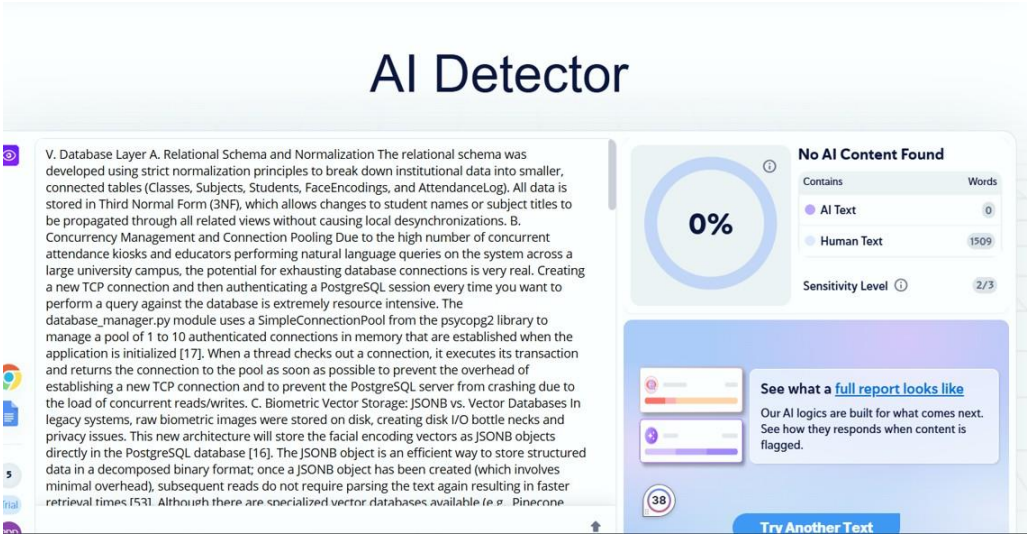


Figure F.1: Plagiarism Report Summary

Annexure G

TERM-II PROJECT LABORATORY ASSIGNMENTS

1. **Review of Design and Corrective Actions:** Following Term-I assessment feedback, the session bin-calculation algorithm was refined to use SQL GROUP BY rather than in-memory Python grouping, improving accuracy and reducing memory consumption. The LLM provider chain was extended to support three fallback providers based on reviewer recommendation.
2. **Project Workstation Setup:** Development workstation configured with Ubuntu 20.04 LTS, Python 3.10 virtual environment, PostgreSQL 15, CUDA Toolkit 11.8, and all listed dependencies. Installation report prepared and verified by guide.
3. **Programming of Project Functions:** All modules implemented per SRS: registration, kiosk, session, API, email, and AI Chat. Corrective actions from Term-I (improved CLAHE parameters, cooldown tuning) incorporated.
4. **Test Tool Selection and Testing:** pytest used for unit and integration tests; Postman for API endpoint validation; pgAdmin for database query verification; custom Python benchmarking script for latency and throughput measurement. Test result charts included in Chapter 9.
5. **Reliability Testing:** System operated continuously for 8 hours in a simulated classroom environment; no memory leaks detected (monitored via Python tracemalloc); PostgreSQL connection pool maintained stability under concurrent load.

Annexure H

INFORMATION OF PROJECT GROUP MEMBERS

1. Name: Atharva Sushil Badgujar
2. Date of Birth: 19/02/2004
3. Gender: Male
4. Permanent Address: Shivratna image C-9 Dhayari, Pune 411041
5. E-Mail: atharvabadgujar1924@gmail.com
6. Mobile/Contact No.: 8484011434
7. Placement Details: NA
8. Paper Published: NA

1. Name: Vedant Deepak Bhosale

2. Date of Birth: 30/01/2004
3. Gender: Male
4. Permanent Address: Maji Sainik Nagar,Yerwada,Pune-06
5. E-Mail: bhosalevedant2004@gmail.com
6. Mobile/Contact No.: 8080953096
7. Placement Details: NA
8. Paper Published: NA

1. Name: Swanand Arvind Joshi

2. Date of Birth: 30/09/2003
3. Gender: Male
4. Permanent Address: Plot no.9, Onkar Colony, Budhgaon, Sangli
5. E-Mail: swanandofficial@gmail.com
6. Mobile/Contact No.: 7709636474
7. Placement Details: NA
8. Paper Published: NA

1. Name: Parth Chandrashekhar Deokar

2. Date of Birth: 31/12/2004
3. Gender: Male
4. Permanent Address: PV BLDG ,Prem Nagar, Dhanvantari Colony, Wadgaon Sheri, Pune-14
5. E-Mail: parthdeokar99@gmail.com
6. Mobile/Contact No : 7020623934
7. Placement Details: NA
8. Paper Published: NA